arXiv:2504.08577v1 [quant-ph] 11 Apr 2025

Extrapolation method to optimize linear-ramp QAOA parameters: Evaluation of QAOA runtime scaling

Vanessa Dehn,[1,*] Martin Zaefferer,[2] Gerhard Hellstern,[3] Florentin Reiter,[1] and Thomas Wellens[1,†]

[1] *Fraunhofer Institute for Applied Solid State Physics IAF, Tullastr. 72, 79108 Freiburg, Germany*
[2] *DHBW Ravensburg, Marienplatz 2, 88212 Ravensburg, Germany*
[3] *DHBW Stuttgart, Herdweg 20, 70174 Stuttgart, Germany*

The Quantum Approximate Optimization Algorithm (QAOA) has been suggested as a promising candidate for the solution of combinatorial optimization problems. Yet, whether - or under what conditions - it may offer an advantage compared to classical algorithms remains to be proven. Using the standard variational form of QAOA requires a high number of circuit parameters that have to be optimized at a sufficiently large depth, which constitutes a bottleneck for achieving a potential scaling advantage. The linear-ramp QAOA (LR-QAOA) has been proposed to address this issue, as it relies on only two parameters which have to be optimized. Based on this, we develop a method to estimate suitable values for those parameters through extrapolation, starting from smaller problem sizes (number of qubits) towards larger problem sizes. We apply this method to several use cases such as portfolio optimization, feature selection and clustering, and compare the quantum runtime scaling with that of classical methods. In the case of portfolio optimization, we demonstrate superior scaling compared to the classical runtime.

I. INTRODUCTION

The quantum approximate optimization algorithm (QAOA) is a widely studied and promising algorithm for the solution of combinatorial optimization problems [1–4], potentially more efficient than classical algorithms in terms of faster runtimes (time to find the optimal solution) or finding solutions of higher quality [5, 6]. However, the standard variational QAOA ansatz, where the variational parameters of the quantum circuits are optimized in connection with a classical optimization routine, is not expected to lead to such a possible runtime advantage. This can be understood from the complexity of optimizing these parameters as follows: the quantum circuit consists of $2p$ circuit parameters $(\vec{\gamma}, \vec{\beta}) = (\gamma_1, ..., \gamma_p, \beta_1, ..., \beta_p)$ that, in standard QAOA, all need to be optimized for a sufficiently large depth p. This requires a high number of circuit evaluations. Additionally, each evaluation itself contains a substantial number of shots (typically around 1000 shots or more), to precisely determine the expectation value of the cost function, which is minimized by the variational algorithm. Furthermore, when using the variational ansatz, the classical optimizer can get stuck in local minima when optimizing $\vec{\gamma}$ and $\vec{\beta}$ [7, 8], especially without a good initial guess. Therefore, fixed schedules, i.e., structured and predefined approaches for selecting circuit parameters, are of interest to circumvent expensive optimization routines [6, 8, 9]. It has been observed that the optimal parameters $(\vec{\gamma}^*, \vec{\beta}^*)$ exhibit patterns, namely that γ_j^* increases smoothly while β_j^* shows a decreasing behavior (with $j = 1, ..., p$) [10]. These patterns are consistent with QAOA being interpreted as realizing a discrete adiabatic schedule with gradually changing unitaries [7].

Based on these patterns, schedules can be formulated by which circuit parameters $(\vec{\gamma}, \vec{\beta})$ are determined for, in principle, arbitrarily high depth p. Examples for such patterns are the Fourier heuristic strategy [10], Trotterized quantum annealing (TQA) [11] or the linear-ramp schedule [7, 12–14], among others [15, 16]. Sometimes (e.g., in [10, 11]), such schedules are used in order to provide initial parameters, which are then further optimized according to standard QAOA. Other works [7, 12–14] propose to use these schedules as fixed schedules, thus saving the enormous overhead involved in further optimizing a large number of circuit parameters, as explained above.

Indeed, a recent study claims that fixed linear-ramp schedules provide a runtime scaling advantage for some combinatorial optimization problems (in particular MaxCut, weighted MaxCut and Max-3-SAT) [17]. Apart from the QAOA depth p, the linear-ramp schedule exhibits only two free parameters Δ_γ and Δ_β (see Sec. IV A below). The authors of Ref. [17] put forward the following conjecture: for suitable values of $\Delta_\gamma, \Delta_\beta$ and QAOA depth chosen as $p = N$, the probability P_{opt} that the QAOA circuit returns the optimal solution remains, on average, constant with increasing problem size N. This conjecture is supported by numerical results obtained on a noiseless simulator with up to $N = 42$ qubits. If true also for larger N, this would imply the possibility of solving these cases of NP-hard problems in polynomial runtime.

An unresolved question concerning this result, however, is how suitable values of the parameters $\Delta_\gamma, \Delta_\beta$ can be found. In our work, we address this question by proposing an extrapolation scheme whereby circuit parameters for larger problem sizes (in our case up to $N = 28$) can be determined on the basis of optimizations performed only on subproblems of smaller size

* vanessa.dehn@iaf.fraunhofer.de
† thomas.wellens@iaf.fraunhofer.de

($N = 4, 6, 8, 10$). With these parameters, we present the scaling of the quantum runtime for three combinatorial optimization problems: portfolio optimization, feature selection and clustering. The runtime is expressed as the median of the total depth of all quantum circuits that have to be executed to find the optimal solution. After comparing the scaling of the quantum runtime with the classical runtime, we find indication of a quantum scaling advantage in the case of portfolio optimization.

Apart from the article [17] mentioned above, similar scaling advantages have recently been found also for Boolean satisfiability problems [18] and the low autocorrelation binary sequences (LABS) problem [19]. In both cases, fixed parameter schedules (though not as simple as linear-ramp) are considered, and the scaling of the QAOA runtime (combined with quantum minimum finding in [19]) is analyzed at fixed QAOA depth p. In our work, we propose to adapt, both, the depth p and also the circuit parameters $(\vec{\gamma}, \vec{\beta})$ with increasing problem size N. We therefore think it is possible that the QAOA runtime scalings obtained in the above mentioned works can be further improved when combining them with ideas presented in this article.

This paper is divided into two main parts, a classical optimization and a quantum optimization part, and is organized as follows: in Sec. II, we give a brief introduction to the combinatorial optimization problems considered in this paper, which are then addressed in Sec. III by purely classical methods. This section provides the investigation as well as the discussion of the scaling behavior of different classical optimization methods (CPLEX, Gurobi, MQLib/Burer2002 Heuristic and Goemans-Williamson algorithm), which is evaluated by the time to find the optimal solution (runtime) as a function of the number of variables (bits). Sec. IV gives an introduction to the linear-ramp QAOA schedule and an explanation of our method to derive suitable QAOA parameters based on extrapolation from smaller towards larger number of qubits. The results are then discussed in Sec. V and we finally conclude in Sec. VI.

II. COMBINATORIAL OPTIMIZATION PROBLEMS

In this section, we introduce three optimization problems that we all formulate as a QUBO [20] problem with the following structure:

$$F(\mathbf{z}) = \sum_{i,j}^{N} F_{ij} z_i z_j + \sum_{i}^{N} f_i z_i \tag{1}$$

with symmetric matrix F, vector f, number of bits N and binary variables $z_1, ..., z_N$.

Taking into account that $z_i z_i = z_i$ (for binary variables $z_i \in \{0, 1\}$) the linear term can be included in the quadratic term, leading to

$$F(\mathbf{z}) = \sum_{i,j}^{N} Q_{ij} z_i z_j \tag{2}$$

where $Q_{ij} = F_{ij} + f_i \delta_{ij}$ is a symmetric matrix defining the problem under consideration.

A. Formulation of the Portfolio Problem

The portfolio optimization problem aims to generate a portfolio of financial assets to maximize the return while minimizing the risk. In order to formulate the problem as a binary problem, we consider the portfolio weights of a stock to be binary and not continuous as in the original formulation by Markowitz [21]. With the aforementioned QUBO function, equation (1), we can directly identify the cost function for the Portfolio optimization problem

$$F_{\text{portfolio}}(\mathbf{z}) = q \sum_{i,j}^{N} z_i z_j \sigma_{ij} - (1-q) \sum_{i}^{N} z_i \mu_i \tag{3}$$

with number of assets N, covariance matrix σ_{ij}, expected return vector μ_i, binary portfolio weights $z_i = 0$ or 1 of stock i, and a risk preference factor q. A budget constraint $B = \sum_i z_i$ indicates the investment of a fixed budget, with B being the number of assets chosen for the portfolio from N total assets. A created portfolio is called feasible if the constraint is fulfilled; otherwise we call a portfolio infeasible. The constraint can be added to the cost function as a penalty term with a prefactor A:

$$F^{(A)}_{\text{portfolio}}(\mathbf{z}) = F_{\text{portfolio}}(\mathbf{z}) + A\Big(\sum_{i=1}^{N} z_i - B\Big)^2. \tag{4}$$

A suitable candidate for the prefactor A can be computed using the routine described in Ref. [22]. In continuation of this paper, we use the annualized returns and covariance matrices for the German DAX 30 (as of 1/2/2021) between 01/01/2016 and 31/12/2020. For a given number of assets, different instances of the problem are obtained by randomly selecting N out of the 30 DAX assets. The risk preference factor q is typically set between 0 and 1 and, in contrast to our typical choice of $q = 1/3$, it is set here to $q = 1$, indicating the focus on the risk part of the problem irrespective of the return part. Concerning the budget, we choose $B = N/2$ (for even N) or $B = (N-1)/2$ (for odd N).

B. Formulation of Feature Selection Problem

Similarly, we can formulate the cost function for the feature selection problem in Machine Learning which can be based on two dependency measures: the first

describes the dependence $\rho_{i,j}$ between the columns i and j of the feature matrix X, and the second is the dependence $\rho_{i,Y}$ between the feature column i and the target vector Y. Furthermore, we consider a set of binary variables $z_i \in \{0,1\}$ for which $z_i = 1$ indicates that the column i is selected in the model, while $z_i = 0$ refers to the column not being selected in the model. An intuitive approach is to select features such that the inter-feature dependence $\sum_{i,j=1}^{N} z_i z_j |\rho_{i,j}|$ is minimized and the dependence between feature and target $\sum_{i=1}^{N} z_i |\rho_{i,Y}|$ is maximized. The cost function can now be constructed as above by combining both criteria:

$$F_{\text{feature}}(\mathbf{z}) = -\Big(\phi \sum_{i=1}^{N} z_i |\rho_{i,Y}| - (1-\phi) \sum_{\substack{i,j=1 \\ i \neq j}}^{N} z_i z_j |\rho_{i,j}|\Big) \tag{5}$$

with weighting parameter $\phi \in [0,1]$, that we set to $\phi = 0.9$. As a dependence measure between the features as well as between features and target we chose to take the linear correlation ρ^{Correl} as proposed in [23]. In accordance to this reference, the "German Credit Data" [24] is used again. It comprises a total of 20 features, encompassing 7 numerical and 13 categorical attributes along with a an imbalanced binary target variable distinguishing non-defaulted cases (700 instances) from defaulted cases (300 instances). After applying one-hot encoding to the categorical features, we end up with a maximum of 48 binary features.

C. Formulation of Clustering Problem

In unsupervised learning, the problem of clustering data is defined as follows: Given a dataset $\mathbf{D}$ containing N data points, where each data point is represented as a feature vector x_i in a feature space $\mathbf{X}$: $D = \{x_1, x_2, ..., x_N\}$, the goal of clustering is to partition the dataset into K clusters, where K is a user-specified or algorithmically determined parameter. Each cluster C_k is a subset of the dataset $\mathbf{D}$ with the following conditions:

1. Completeness: Every data point x_i belongs to exactly one cluster C_k, and the union of all clusters covers the entire dataset.

2. Homogeneity: Data points within the same cluster are similar to each other, and data points in different clusters are dissimilar. This is typically measured using a distance or similarity metric.

3. Minimization Objective: The clusters are formed in a way that minimizes an objective function. This objective function may vary depending on the specific clustering algorithm being used.

Mathematically, the clustering problem can thus be defined as finding a partition of the dataset into K clusters that optimizes an objective function J, where:

$$J = f(D, C_1, C_2, ..., C_K). \tag{6}$$

The objective function J can be chosen based on the specific clustering algorithm's criteria, such as minimizing intra-cluster distances or maximizing inter-cluster distances. The goal of clustering is to discover meaningful patterns or groupings within the data without any prior knowledge of the class labels. Various clustering algorithms, such as K-means [25], hierarchical clustering [26], and DBSCAN [27], can be used to solve this problem of finding the optimal partition of the dataset into clusters based on different criteria and distance measures.

The Euclidean distances for all data pairs x_i, x_j, i.e.

$$w_{ij} = ||x_i - x_j||_2 \tag{7}$$

can be interpreted as the elements of a symmetric matrix W, where all diagonal elements are zero. This matrix W can now be interpreted as an adjacency matrix of a graph G, where each vertex represents an element of W, and w_{ij} is the weight of the edge between the vertices i and j. In general, the graph is fully connected.

The common assumption in clustering – distant points belonging to different clusters – translates in the case of $K = 2$ clusters to the problem of maximizing the overall sum of all weights (distances) between nodes with different labels. Therefore, we come to a Maximum-Cut (Maxcut) problem - Maxcut(G, W) for the dense graph G of the distance matrix W [28, 29]. A cut is a set of edges that separates all vertices V into two disjoint sets S and $\bar{S}$ with $S \cup \bar{S} = V$. The cost of a cut is given by the sum of all weights of edges connecting vertices in S with vertices in $\bar{S}$. With the introduction of binary variables $z_i \in \{0,1\}, i = 1, ..., N$ which denote whether x_i belongs to S or $\bar{S}$, the sum of all cuts can be expressed as the cost function

$$F_{\text{clustering}}(\mathbf{z}) = \sum_{i,j=1}^{N} w_{ij} \cdot (z_i + z_j - 2 \cdot z_i z_j). \tag{8}$$

Again, we end up with a QUBO problem. Here, we decided to use the "Moons" and "Blobs" datasets of Scikit-Learn [30] with two features, i.e., $x \in \mathbb{R}^2$. Each data point then induces one binary decision variable. Using Moons and Blobs have the advantage that the clustering results can be either checked by e.g. the K-Means algorithm or visually.

III. CLASSICAL OPTIMIZATION

To solve the optimization problems introduced above, we use the following methods/software solutions:

- CPLEX [31]
- Gurobi [32]

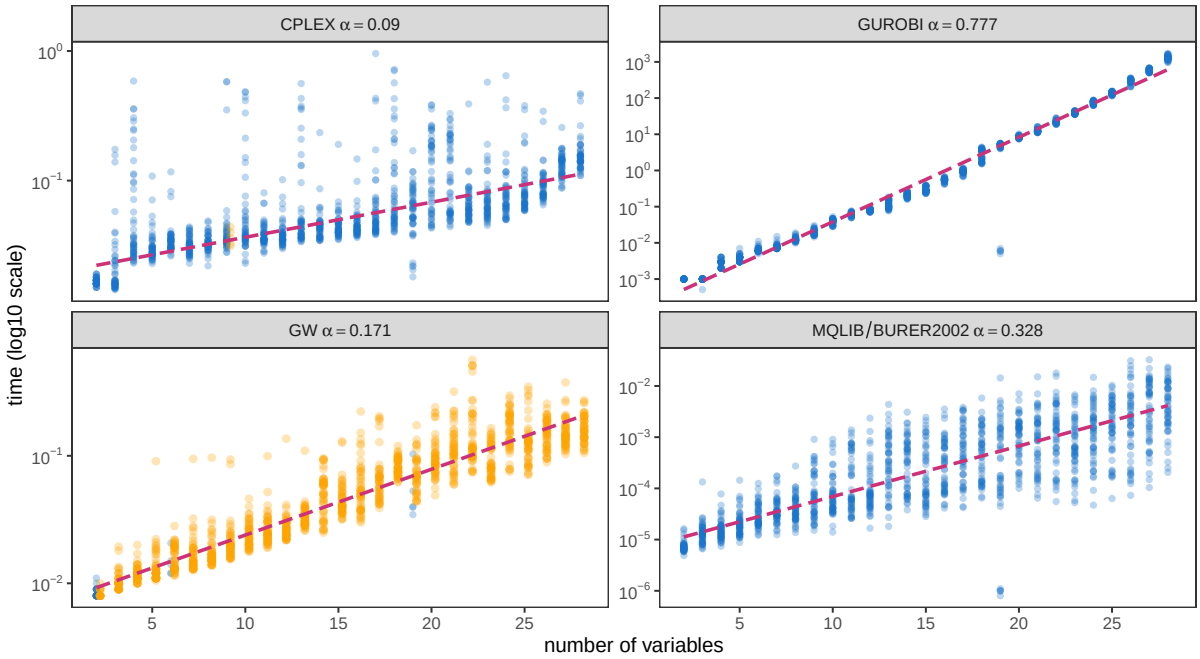

FIG. 1. Portfolio optimization: Runtime T (seconds) obtained with CPLEX, Gurobi, Goemans-Williamson and MQLib/Burer2002, as a function of the number N of variables (bits). The dots represent the results for the ten problem instances, with blue indicating it was optimally solved, while orange shows an approximate final solution. The slope α of a robust log2-linear model, i.e., $T \propto 2^{\alpha N}$ (red dashed line) is denoted in the header of each figure.

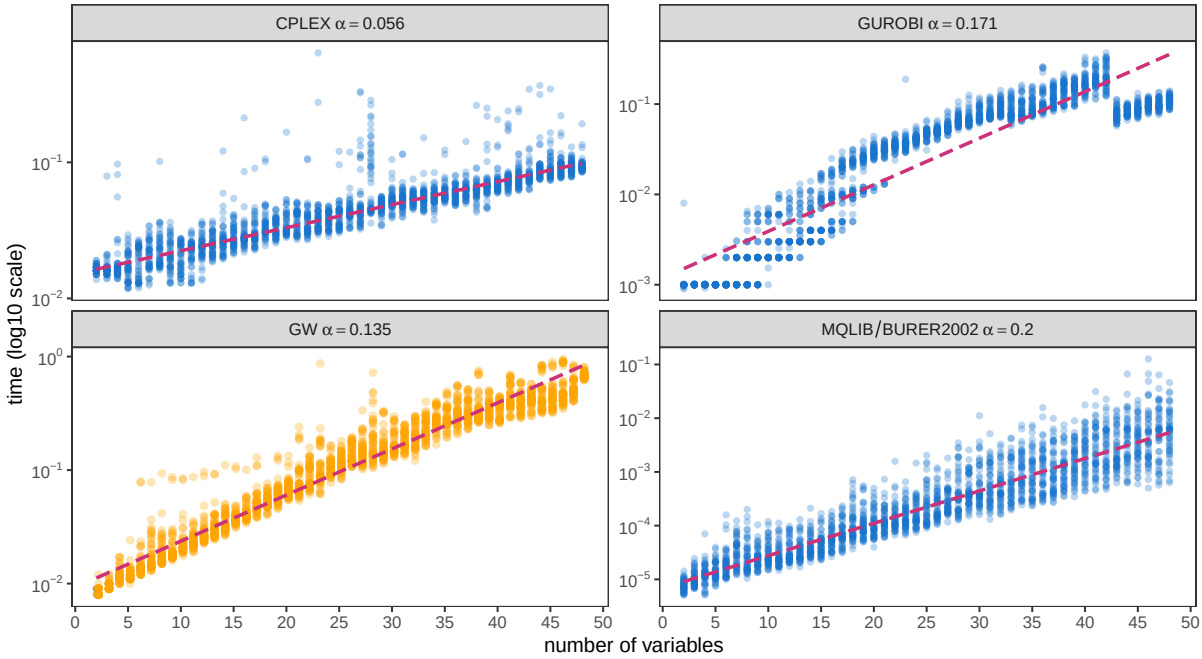

FIG. 2. Feature Selection: Runtime T (seconds) obtained with CPLEX, Gurobi, Goemans-Williamson and MQLib/Burer2002, as a function of the number N of variables (bits). The dots represent the results for the ten problem instances, with blue indicating it was optimally solved, while orange shows an approximate final solution. The slope α of a robust log2-linear model, i.e., $T \propto 2^{\alpha N}$ (red dashed line) is denoted in the header of each figure.

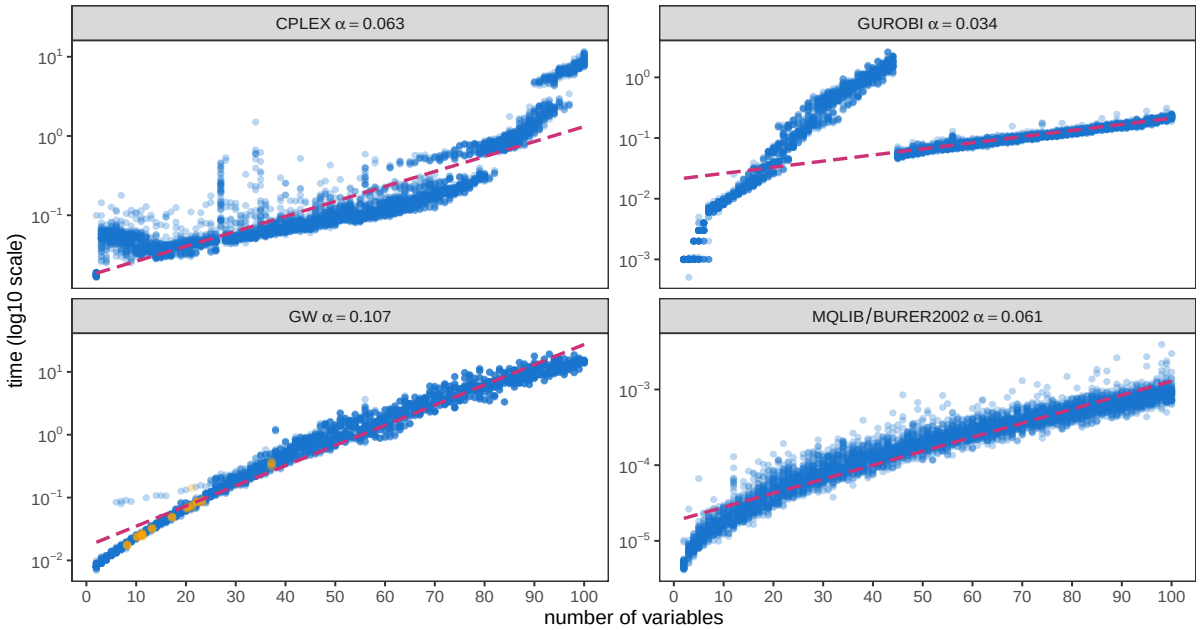

FIG. 3. Clustering (Moons): Runtime T (seconds) obtained with CPLEX, Gurobi, Goemans-Williamson and MQLib/Burer2002, as a function of the number N of variables (bits). The dots represent the results for the ten problem instances, with blue indicating it was optimally solved, while orange shows an approximate final solution. The slope α of a robust log2-linear model, i.e., $T \propto 2^{\alpha N}$ (red dashed line) is denoted in the header of each figure.

- MQLib/Burer2002 Heuristic [33]
- Goemans-Williamson's algorithm [34]

Apart from MQLib/Burer2002, these classical optimization methods are all available in IBM's Qiskit [35] for solving QUBOs with a uniform function call. Due to the large number of all possible solutions (2^N), a brute-force approach is infeasible for large N. Note that Goemans-Williamson's is an approximative algorithm, it does not guarantee to find the global optimum. Strictly speaking, none of the algorithms guarantees to find the global optimum, although at least CPLEX and Gurobi may provide additional information about the optimality of solutions. That is, they compute upper and lower bounds for the objective function value, which may indicate global optimality if the difference between both bounds is zero. MQLib/Burer2002, finally, does not provide information about the optimality of the solution. It continues trying to find better solutions until a user-defined maximum runtime is reached. At the end, it returns these solutions together with the times at which they have been found.

To investigate the scaling behavior of the different classical optimization methods, we perform the following experiment:

1. Sample ten instances with size N randomly from the data set, for each combinatorial optimization problem.

2. For each instance, we measure five times how long each algorithm takes to find a solution. The five-fold repetition is intended to account for random variations of the runtime.

3. Repeat this for each N, which varies from 2 to N_{max}, where N_{max} depends on the problem considered.

In the figures 1 - 3 we show the runtime measured during execution as function of the number N of variables and the slope of the fit through the data. The slope is an estimator of the scaling factor which depends on the data set and on the algorithm.

In Fig. 1 the results for the portfolio optimization problem are shown. The absolute values for the runtime differ strongly between the four classical optimization schemes. On the one hand, Gurobi shows the longest absolute runtimes as well as the highest scaling factor. On the other hand, CPLEX shows the lowest scaling factor and MQLib/Burer2002 the shortest absolute runtime.

A similar behavior can be observed for the feature selection problem, see Fig. 2. Again, CPLEX has the lowest scaling factor, MQLib/Burer2002 the shortest absolute runtimes, and Goemans-Williamson's algorithm does not find the optimal solution in most cases. However, MQLib/Burer2002 now has the highest scaling factor, and GW the longest absolute runtimes.

In Fig. 3 the clustering results for the Moons dataset are shown. Interestingly, GW mostly finds the optimal solutions for this problem class, and Gurobi now has the lowest scaling factor, albeit also showing a strong discontinuity of runtimes at about $N = 45$. Similarly, but less

pronounced, also CPLEX displays jumps of the runtime, e.g., around $N = 90$. GW has the largest scaling factor in this case. The results for the Blobs dataset are depicted in Fig. 13 which can be found in the Appendix A.

In summary, it can be concluded that the various classical optimization methods exhibit different scaling behaviors on the investigated combinatorial optimization problems. For the subsequent comparison with quantum algorithms, we will utilize the MQLib/Burer2002 algorithm for the following reasons: The Goemans-Williamson's algorithm is unsuitable for the comparison because it frequently does not find the optimum. The commercial optimization algorithms CPLEX and Gurobi are problematic because their search approach may change unexpectedly depending on various criteria, as is also evidenced by the jumps and discontinuities of their runtime graphs, making it very hard to model their behavior. For these reasons, the MQLib/Burer2002 algorithm – which, in addition, displays the fastest runtimes – appears to be the most suitable for comparison with the quantum algorithm.

IV. QUANTUM OPTIMIZATION

A. Linear-ramp QAOA

When using QAOA to solve the above introduced optimization problems on a quantum computer, the following parameterized quantum state is generated:

$$|\psi_{\vec{\gamma},\vec{\beta}}\rangle = \hat{U}_M(\beta_p)\hat{U}_F(\gamma_p)...\hat{U}_M(\beta_1)\hat{U}_F(\gamma_1)\,|\psi_0\rangle_M \tag{9}$$

with circuit parameters $\vec{\gamma} = (\gamma_1, ..., \gamma_p)$ and $\vec{\beta} = (\beta_1, ..., \beta_p)$, mixer unitary $\hat{U}_M(\beta_j) = e^{-i\beta_j \hat{M}}$ with transverse-field mixer $\hat{M} = -\sum_{i=1}^{n} \hat{X}_i$, and cost unitary $\hat{U}_F(\gamma_j) = e^{-i\gamma_j \hat{F}}$ with cost Hamiltonian $\hat{F}$ (defined by $\hat{F}|\mathbf{z}\rangle = F(\mathbf{z})|\mathbf{z}\rangle$ for basis states $|\mathbf{z}\rangle$). The initial state $|\psi_0\rangle_M$ corresponds to the mixer Hamiltonian $\hat{M}$, or more precisely, it is its eigenstate associated with its smallest eigenvalue. Hence, the state is the equal superposition state $|\psi_0\rangle = |+\rangle^{\otimes N}$ for this choice of $\hat{M}$. Finally, the probability of measuring the optimal solution $\mathbf{z}_{\text{opt}}$ is obtained as:

$$P_{\text{opt}} = \left|\langle \mathbf{z}_{\text{opt}}|\psi_{\vec{\gamma},\vec{\beta}}\rangle\right|^2 \tag{10}$$

In contrast to standard QAOA, where suitable values for the circuit parameters are suggested through a classical optimization routine, we sample them directly from the ansatz

$$\gamma_j = \Delta_\gamma x_j \tag{11}$$
$$\beta_j = \Delta_\beta (1 - x_j) \tag{12}$$

with

$$x_j = \frac{j - \frac{1}{2}}{p}, \quad j = 1, 2, \ldots, p, \tag{13}$$

where the parameters $\Delta_\gamma, \Delta_\beta$ are left to be optimized. Additionally, we find optimized values for p, with the procedure outlined in more detail in Sec. IV B 4. Those three parameters characterize the linear-ramp schedule which is illustrated in Fig. 4.

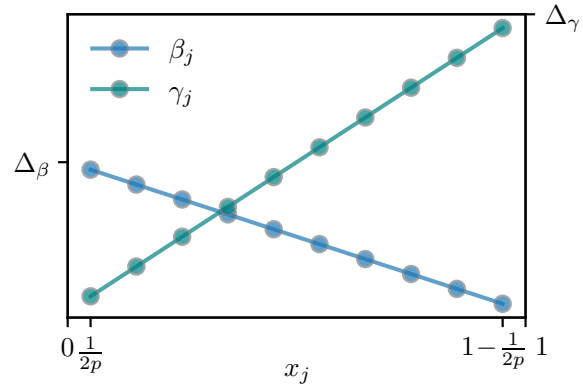

FIG. 4. Linear-Ramp protocol according to Eqs. (11-13) (illustrated here for $p = 10$).

B. LR-Extrapolation

We follow the idea to discard the variational optimization of the circuit parameters and finding suitable values for $\Delta_\gamma, \Delta_\beta$ and p by extrapolating from smaller to larger problem sizes, corresponding to the number of qubits N. This enables us to make a prediction of the circuit parameters for larger problem sizes, while only the optimization over small problem sizes has to be performed. The method consists of three parts: the first part includes a reduction of the original problem to smaller subproblems, the second part describes the extrapolation, and the third part focuses on the post-processing of the results.

1. *Random Sampling and Grid Search*

For each QUBO instance of size $N \in \{12, ..., 28\}$, we first create random sub-instances of size $\tilde{N} \in \{4, 6, 8, 10\}$. This is done by randomly selecting $\tilde{N}$ out of the N binary variables and then constructing the QUBO cost functions as described in Sec. II. For Feature Selection and Clustering, this simply amounts to choosing the corresponding sub-matrices of Q_{ij}, see Eq. (2). For Portfolio Optimization, the dependence of the budget constraint B on the problem size is taken into account (whereas the penalty term A is kept constant), see Eq. (4). Thereby, 10 random sub-instances are created for each $\tilde{N}$.

In the next step, we use the 10 sampled sub-instances of the smallest size ($\tilde{N} = 4$) and perform a grid search with an initial depth value $p_0 = 6$. That is, we construct

a grid of $\log(\Delta_\gamma)$, $\log(\Delta_\beta)$ points, compute the probability of finding the optimal solution P_{opt} for each instance and determine the average probability $\overline{P_{\text{opt}}}$ (averaged over the 10 sub-instances). For all subproblem sizes, we choose a 11×11 grid with $\log(\Delta_\gamma)_{\min} \leq \log(\Delta_\gamma) \leq \log(\Delta_\gamma)_{\max}$ and $\log(\Delta_\beta)_{\min} \leq \log(\Delta_\beta) \leq \log(\Delta_\beta)_{\max}$. The initial size of the grid (for $\tilde{N} = 4$) is chosen as $\log(\Delta_\gamma)_{\min} = -1, \log(\Delta_\beta)_{\min} = -1.5$ and $\log(\Delta_\gamma)_{\max} = +1$ and $\log(\Delta_\beta)_{\max} = +0.5$ The logarithmic scale, i.e., considering $\log(\Delta_{\gamma,\beta})$ instead of $\Delta_{\gamma,\beta}$, is used to cover a sufficiently wide range of values. (Here and in the following, logarithms are taken with respect to basis 10.)

2. *Multivariate Skew Gaussian Fit and Grid Reduction*

The above grid search provides a two-dimensional landscape consisting of $11 \times 11 = 121$ data points, see Fig. (5) as an example (for sub-problem size $\tilde{N} = 10$ and depth $p = 107$). We can clearly see a dark blue region where the average probability $\overline{P_{\text{opt}}}$ assumes the largest values. It is the center of this region that we are interested in and which we will use for our extrapolation procedure below.

In order to determine the position of the center, we fit the discrete 11×11-landscape by a continuous function. For this purpose, we choose a multivariate skew Gaussian function of the following form:

$$f_{\boldsymbol{\mu},\Sigma,\boldsymbol{\alpha},A,B}(\mathbf{x}) = A \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \times \frac{1}{2}\left[1 + \operatorname{erf}\left(\boldsymbol{\alpha}\cdot\frac{\mathbf{x}-\boldsymbol{\mu}}{\sqrt{2}}\right)\right] + B \quad (14)$$

where $\mathbf{x} = \{\log(\Delta_\gamma), \log(\Delta_\beta)\}$. The fit parameters consist of a two-dimensional vector $\boldsymbol{\mu}$ (center of the Gaussian function), a 2×2 (inverse) covariance matrix $\Sigma^{-1} \geq 0$, a two-dimensional vector $\boldsymbol{\alpha}$ characterizing the skewness and two parameters $A, B \geq 0$. The maximum of this function is determined numerically and provides us with $\{\log(\Delta_{\gamma,\text{opt}}), \log(\Delta_{\beta,\text{opt}})\} = \mathbf{x}_{\text{opt}}$ and $\overline{P_{\text{opt}}}\big|_{\max} = f_{\boldsymbol{\mu},\Sigma,\boldsymbol{\alpha},A,B}(\mathbf{x}_{\text{opt}})$. Additionally, we extract the widths $s_\gamma = \sqrt{\Sigma_{1,1}}$ and $s_\beta = \sqrt{\Sigma_{2,2}}$ of the blue region from the diagonal elements of the covariance matrix Σ.

With these parameters, we set the new coordinates for the grid search of the next-sized subproblem in the following way

$$\begin{aligned} \log(\Delta_\gamma)_{\min} &= \log(\Delta_{\gamma,\text{opt}}) - s_\gamma \\ \log(\Delta_\gamma)_{\max} &= \log(\Delta_{\gamma,\text{opt}}) + s_\gamma \\ \log(\Delta_\beta)_{\min} &= \log(\Delta_{\beta,\text{opt}}) - s_\beta \\ \log(\Delta_\beta)_{\min} &= \log(\Delta_{\beta,\text{opt}}) + s_\beta \end{aligned} \quad (15)$$

which ensures that the optimum of the previous-sized problem is at the center of the following grid. This procedure is then repeated until we reach sub-problem size $\tilde{N} = 10$.

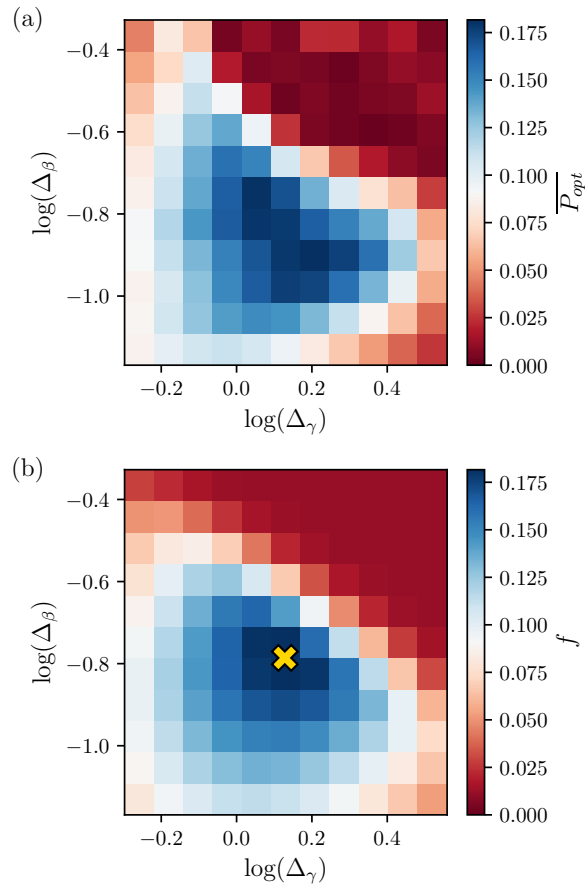

FIG. 5. (a) Portfolio Optimization: 2-dimensional grid search landscape for 10 sub-problems of size $\tilde{N} = 10$ sampled from a problem of size $N = 28$ at depth $p = 107$. The blue region shows the region with highest average ground state probability $\overline{P_{\text{opt}}}$. (b) Skewed Gaussian fit f, see Eq. (14), to the data shown in (a). The optimal values ($\log(\Delta_{\gamma,\text{opt}})$, $\log(\Delta_{\beta,\text{opt}})$) are represented by the yellow cross.

3. *Extrapolation*

After having found optimized parameters $\{\log(\Delta_{\gamma,\text{opt}}), \log(\Delta_{\beta,\text{opt}})\}$ and $\overline{P_{\text{opt}}}\big|_{\max}$ for all sizes $\tilde{N} = 4, 6, 8, 10$ of the sub-instances, we extrapolate these values towards the size N of the original instance. For this purpose, we assume a linear behaviour on a log-log scale, i.e., we fit functions $y_i = a_i x + b_i$ to the above data points, where $x = \log(\tilde{N})$, $y_1 = \log(\Delta_{\gamma,\text{opt}})\}$, $y_2 = \log(\Delta_{\beta,\text{opt}})$ and $y_3 = \log\left(\overline{P_{\text{opt}}}\big|_{\max}\right)$. Thereby, we obtain extrapolated values $\Delta_{\gamma,\text{extr}}$, $\Delta_{\beta,\text{extr}}$ of the linear-ramp circuit parameters together with an estimation $P_{\text{opt,extr}}$ of the probability to measure the optimal solution of our problem instance with size N. In Fig. 6, this extrapolation procedure is illustrated for four different portfolio optimization instances of size $N = 28$, each at a certain depth p (128 ,215, 304 and 608). Each depth corresponds to the optimal depth for that

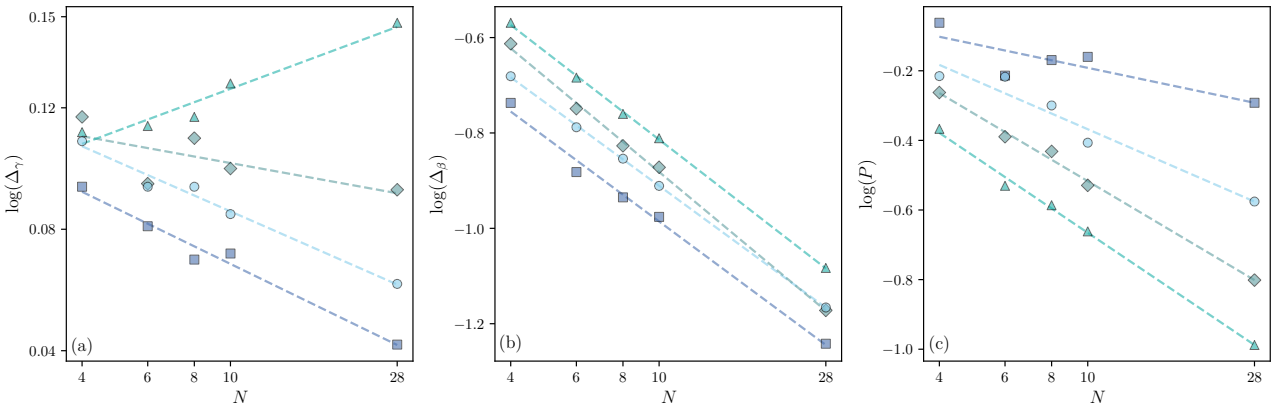

FIG. 6. Extrapolation of (a) $\log(\Delta_\gamma)$, (b) $\log(\Delta_\beta)$ and (c) $\log(P_{\rm opt})$ for four instances of the portfolio optimization problem with $N = 28$ assets (triangles, diamonds, circles, squares). The symbols at $N = 4, 6, 8, 10$ indicate the optimized values found for the respective subproblems. The dashed lines represent linear fits (on log-log scale) through these points. Finally, the markers at $N = 28$ display the extrapolated values of these parameters. Each instance is processed at a certain depth: $p = 128$ (triangles), $p = 215$ (diamonds), $p = 304$ (circles), $p = 608$ (squares).

instance, as determined in Sec. IV B 4 below.

To check the quality of our extrapolation, we have determined a 20×20 grid search landscape for one of the four instances shown in Fig. 6. Note that this grid search is numerically quite expensive, since it involves simulations of large quantum circuits (with $N = 28$ qubits) – in contrast to the extrapolation procedure based on simulations of small circuits (with $\tilde{N} = 4, 6, 8, 10$). As shown in Fig. 7, the estimated values $\Delta_{\gamma,\rm extr}$, $\Delta_{\beta,\rm extr}$ obtained from the extrapolation agree quite well with the true maximum. This example suggests that the predictions made using the extrapolation method provide a good estimate for the linear-ramp parameters. More data supporting this conclusion (at least for most instances) will be shown in Sec. V.

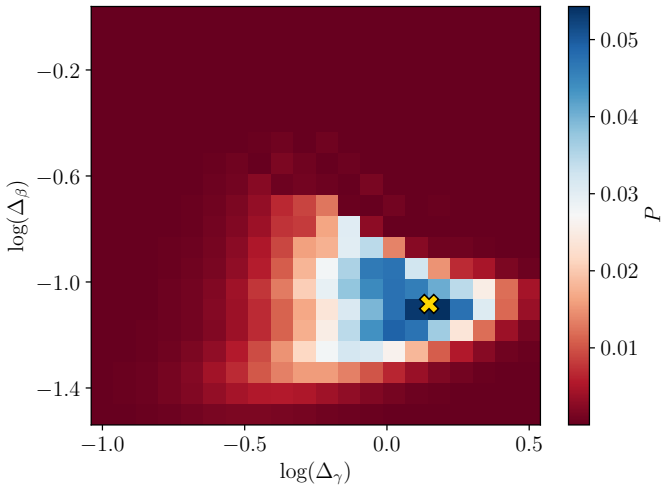

FIG. 7. Grid search landscape for one instance (corresponding to triangles in Fig. 6) with $N = 28$ qubits and QAOA depth $p = 128$. The optimal region is visualized by the dark blue region. The optimal $(\Delta_{\gamma,\rm extr}, \Delta_{\beta,\rm extr})$ configuration predicted by the extrapolation method, represented by the yellow 'x' symbol, is close to the optimal configuration found through grid search. Moreover, the corresponding maximum value of $P_{\rm opt} \simeq 0.055$ (see the labels of the color bar on the right-hand side) is of the same order of magnitude as the extrapolated value of $P_{\rm opt,extr} \simeq 0.103$. This indicates that the extrapolation method performs well.

4. *Total depth and optimization of p*

As described above, the extrapolation procedure provides us with suitable estimates of the linear-ramp circuit parameters $\Delta_{\beta,\gamma}$ and the corresponding probability $P_{\rm opt}$ of measuring the optimal solution. What remains is to choose an appropriate value of the QAOA depth p. In principle, we may increase $P_{\rm opt}$ by increasing p, which, however, leads to longer circuits. To find an optimal value of p, we want to minimize the runtime of our quantum algorithm until the optimal solution is found. For this purpose, we concentrate in the following on the depth d of the QAOA quantum circuit. The corresponding time is then given by multiplying the depth with the time of a single quantum gate. Depending on the given quantum hardware, more specific estimates could be obtained, e.g., by taking into account different times of single-qubit and two-qubit gates or times of initial state preparations and final state measurements. We expect, however, that these details will not significantly affect the scaling behaviour in the limit of large problem sizes.

To determine the total quantum runtime, we therefore multiply the depth d of a single QAOA circuit (which, in turn is roughly proportional to the QAOA depth p) with the number $N_{\rm shots}$ of shots required to find the optimal solution:

$$D = d \cdot N_{\rm shots} \tag{16}$$

The median of the latter (defined by the condition that the probability to find the optimal solution after N_{shots} circuit executions equals 1/2) is given by:

$$N_{\text{shots}} = \frac{\log(\frac{1}{2})}{\log(1 - P_{\text{opt}})} \tag{17}$$

Alternatively, the expectation value given by $1/P_{\text{shots}}$ could also be used instead of the median.

The optimal QAOA depth p_{opt} can now be derived by minimizing D. More precisely, we minimize the estimated value of D based on the above extrapolation procedure, i.e., we set $P_{\text{opt}} = P_{\text{opt,extr}}$ in Eq. (17). We consider a logarithmically spaced grid of p-values $p \in \{1, 2, 3\} \cup \{\lfloor 2^{i/4} \rfloor,\ i = 9, 10, 11, \dots\}$ and, starting from an initial value p_0 (for which we take the optimal value found for the previously calculated instance, or $p_0 = 6$ in case of the first instance corresponding to the smallest problem size $N = 12$), we increase or decrease p on this grid until a local minimum of D is found.

V. RESULTS AND DISCUSSION

In this section, we analyze the LR-QAOA runtime scaling for the three use cases (portfolio optimization, feature selection and clustering) introduced in Sec. II. We increase the problem size from $N = 12$ to $N = 28$. For each N, 10 randomly chosen problem instances are generated, and LR-QAOA circuit parameters (Δ_γ, Δ_β and p) are found using the extrapolation procedure described in Sect. IV A. Finally, the total depth D – which is proportional to the quantum runtime for finding the optimal solution (see Sec. IV B 4) – is determined by Eqs. (9,10,16) and (17) (using a noiseless quantum simulator). Note that the last step is the only one which involves the execution of a quantum circuit with larger size ($N > 10$). Finally, we determine the runtime scaling with increasing N and compare it with the classical result (MQLib/Burer2002) of Sec. III.

A. LR-QAOA runtime scaling

For the case of portfolio optimization, the results for the individual 10 random instances per N are displayed by the blue circles in Fig. 8. To extract the scaling behaviour, we perform an exponential fit, $D \propto 2^{\alpha N}$ (red dashed line). With this, we find a scaling exponent for the runtime of $\alpha \approx 0.22$, which is smaller than the classical scaling exponent of $\alpha \approx 0.328$ (see Fig. 1 bottom right). This indicates a potential quantum advantage regarding the runtime scaling.

In order to test our extrapolation procedure, we further optimized the circuit parameters (Δ_γ, Δ_β and p) – starting from the extrapolated parameters as initial points – by directly minimizing the actual total depth D (instead of its estimate based on extrapolation). Since this involves many executions of large quantum circuits (up to

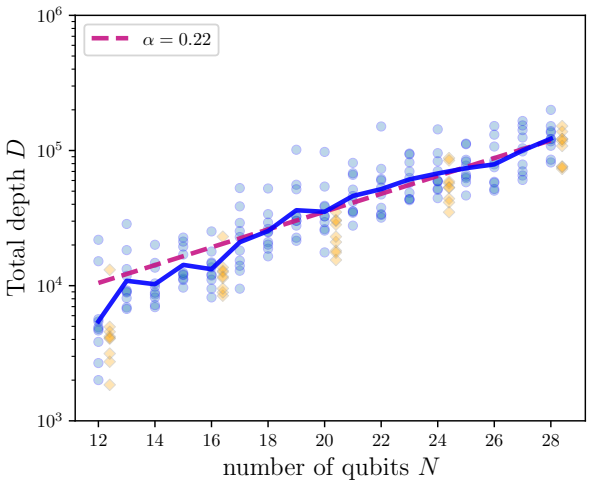

FIG. 8. Portfolio Optimization: Scaling of quantum runtime expressed as total depth D, see Eq. (16), with increasing problem size N (number of qubits) using linear-ramp QAOA with circuit parameters determined from extrapolation. The red dashed line represents an exponential fit with slope α (i.e. $D \propto 2^{\alpha N}$) of the geometric mean (blue line) over 10 problem instances (transparent blue dots). Compared to the classical runtime $\alpha = 0.328$, see Fig. 1 (MQLib/Burer2002), the increase of the quantum runtime is slower, thus indicating a potential quantum scaling advantage. The yellow diamond shapes represent the optimal values found by a minimizer, starting from the extrapolated parameters as initial point. For better visibility, the diamonds have been shifted along the x-axis.

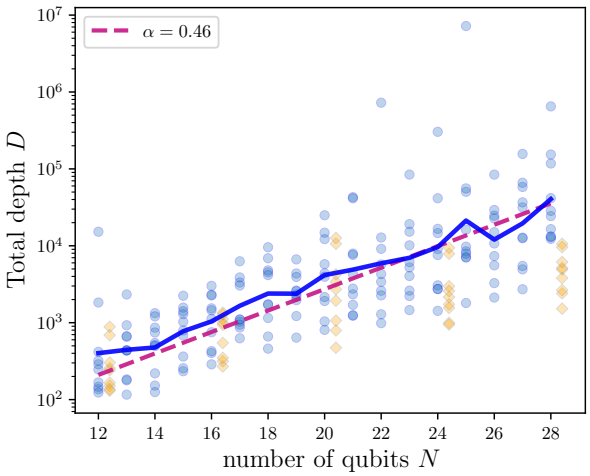

FIG. 9. Feature Selection: Scaling of the quantum runtime represented in the same way as for portfolio optimization (see Fig. 8). Utilizing our LR-QAOA variant results in a faster increase in runtime, see exponential fit with slope $\alpha \simeq 0.46$ (red dashed line), in comparison to classical methods ($\alpha \simeq 0.2$, see Fig. 2, bottom right). Hence, in contrast to portfolio optimization, we see no indication of a quantum advantage for feature selection.

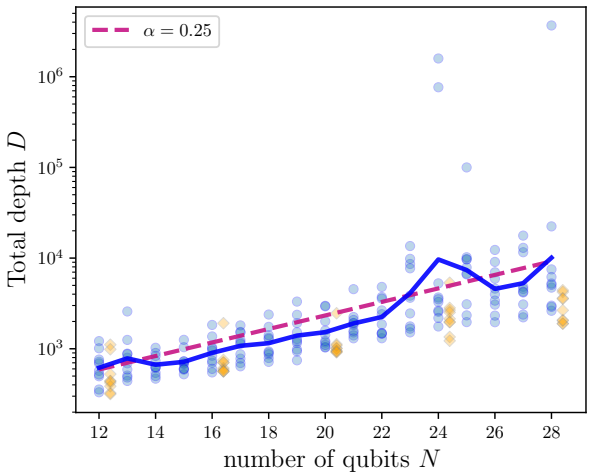

FIG. 10. Clustering (Moons): Scaling of the quantum runtime represented in the same way as for portfolio optimization and feature selection (Figs. 8 and 9). Similar as in the case of feature selection, our method ($\alpha \simeq 0.25$) does not provide an advantage for this optimization problem over our classical results ($\alpha = 0.061$, see Fig. 3).

$N = 28$) for each instance, we restricted ourselves to every fourth problem size $N = 12, 16, \ldots$. All in all, we see that the depths D achieved with the extrapolated parameters (blue circles) are remarkably close to the actual minima (yellow diamonds). This underlines the effectiveness of the extrapolation scheme.

Fig. 9 shows the results for the feature selection problem, which we evaluate on the same footing as portfolio optimization. Here, we find a scaling for the quantum runtime of $\approx 2^{0.46N}$. In comparison, the classical runtime exhibits a scaling of $\approx 2^{0.2N}$ (see Fig. 2, bottom right) indicating that this combinatorial optimization problem is faster to solve classically. Furthermore, we observe some outliers with extremely large values of D (e.g., one instance with $D \approx 10^7$ at $N = 25$). This suggests that, for the feature selection problem, the extrapolation scheme is less stable than in the case of portfolio optimization and fails in providing good circuit parameters for some instances.

A similar picture is observed for the clustering problem (Moons), as shown in Fig. 10. In this case, we observe a quantum runtime scaling of $\approx 2^{0.25N}$. In comparison, the classical heuristic MQLib/Burer2002 exhibits a scaling of $\approx 2^{0.061N}$ (see Fig. 3, bottom right), indicating that the quantum algorithm takes significantly longer to solve the problem. For most instances, we find the minimized values of D (yellow dots at $N = 12, 16, \ldots$) not far below the values achieved by the extrapolated parameters (blue dots), indicating that – apart from a few outliers at $N = 24, 25$ and 28 – our extrapolation method provides good suggestions for Δ_γ and Δ_β, although it does not lead to an overall better scaling compared to the classical runtime.

B. Discussion of Scaling Conjecture

In this section, we discuss the scaling regarding the conjecture that is proposed in [17]. It states that when there is no high concentration of solutions close to the optimal one, then there are LR-QAOA parameters $\Delta_\gamma, \Delta_\beta$ leading to a scaling of the success probability as $P_{\text{opt}} = 1/2^{\eta N/p}$ where N is the number of qubits, p the QAOA depth and η a constant. According to this conjecture, a constant success probability can be maintained for increasing problem size N when choosing the QAOA depth p proportional to N. This would open the exciting possibility of solving the combinatorial optimization problem in polynomial instead of exponential time.

In our case of portfolio optimization (on which we focus in the following since, as shown above, it is the only one with an indication of a possible quantum advantage), the parameters $\Delta_\gamma, \Delta_\beta$ extracted from the extrapolation scheme do not turn out be constant, but exhibit an approximately algebraic dependence on the problem size N, see Fig. 11(a,b). Furthermore, the QAOA depth p does not linearly increase with N, but approximately follows $p \propto N^3$, see Fig. 11(c). Remarkably, however, we see in Fig. 12 that the success probability P_{opt} achieved by these parameters remains approximately independent of N. This would imply that the quantum runtime expressed in terms of the total depth D scales polynomially, i.e., $D \propto N^4$ (since the depth d of a single QAOA circuit scales like $d \propto pN$, and $p \propto N^3$ as shown above). Hence, in order to answer the important question of exponential vs. polynomial scaling, future investigations with a larger range of problem sizes (going beyond $N = 28$) are required.

VI. CONCLUSION

In this work, we present an extrapolation method to optimize linear-ramp QAOA parameters, enabling us to discard the need for variational optimization employed in standard QAOA. We start with the introduction of three different combinatorial optimization use cases: Portfolio Optimization, Feature Selection, and Clustering, which are subsequently solved using classical methods (CPLEX, Gurobi, MQLib Burer2002, Goemans-Williamson) to assess the classical runtime. The scaling of the runtime is investigated by fitting the runtime with an exponential model, which serves as a benchmark for the runtimes of our quantum algorithm. In particular, we concentrate on the runtime scaling of the MQLib Burer2002 heuristics, since it exhibits the lowest absolute values of the runtime together with a well-defined scaling behaviour over a large range of problem sizes.

Our quantum algorithm is based on linear-ramp QAOA, for which the circuit parameters are determined by a linear ansatz. This ansatz is characterized by three parameters (Δ_γ, Δ_β and depth p), which have to be optimized. In this paper, we expose a method to derive

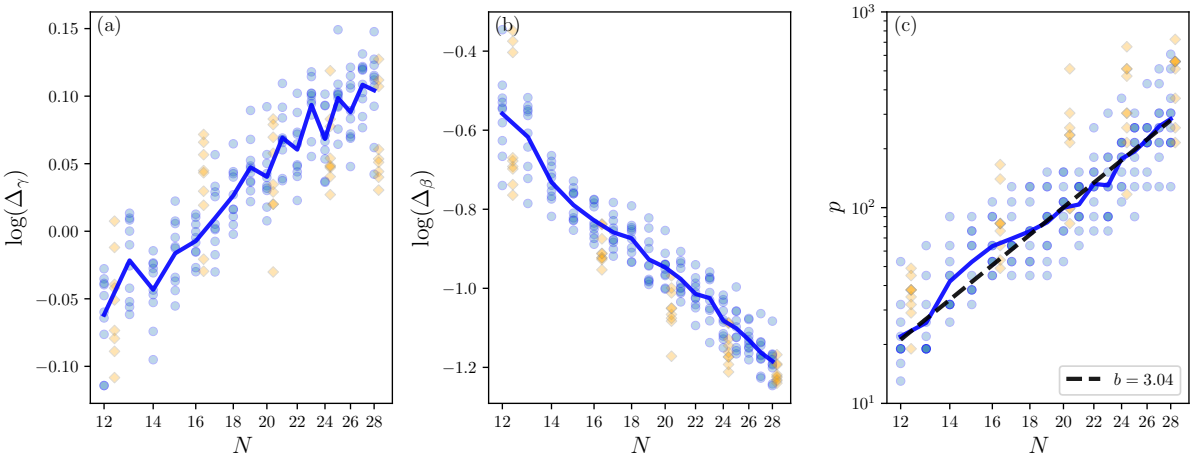

FIG. 11. Portfolio Optimization: optimized linear-ramp parameters (a): Δ_γ, (b): Δ_β, and (c): QAOA depth p as a function of number of qubits N, depicted as a log-log plot, for the same instances as in Fig. 8. The transparent blue dots represent values obtained from extrapolation and the blue line shows the geometric mean over all instances. The values for Δ_γ as well as for p are increasing with increasing N while the values for Δ_β are decreasing. The yellow diamonds represent the optimal values found by a minimizer (as in Fig. 8) and have been shifted along the x-axis for better visibility. In (c), a fit of the geometric mean is generated with the function $p = a \cdot N^b$ (black dashed line) and scales as $\propto N^3$.

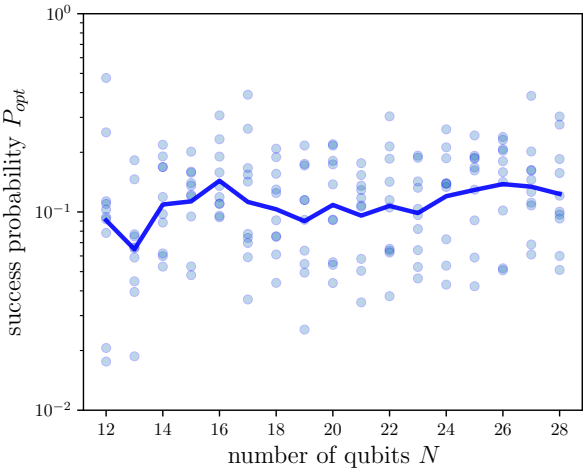

FIG. 12. Portfolio Optimization: Success probability P_{opt} of finding the optimal solution for the same parameters as in Figs. 8 and 11 (blue dots) with geometric mean (blue line) as a function of the problem size N. The success probability remains roughly constant (around $P_{\text{opt}} \simeq 0.1$) for increasing problem size N.

those optimized parameters through extrapolation from smaller to larger problem sizes. In particular, the extrapolation is based on optimizations of QAOA circuits with reduced size ($\tilde{N} = 4, 6, 8, 10$) corresponding to randomly sampled sub-instances of the original problem instance (with size N up to 28). Finally, the quantum runtime is expressed in terms of the total depth $D = dN_{\text{shots}}$, i.e., the depth d of a single linear-ramp QAOA circuit times the (median of) the number of shots required to measure the optimal solution on a noiseless simulator. Comparing quantum and classical runtime scalings, we find an indication of quantum advantage for one of the three use cases mentioned above, i.e., for portfolio optimization.

One might argue that the cost for finding the optimized circuit parameters should also be taken into account in the evaluation of the quantum runtime scaling. Although, as mentioned above, our extrapolation scheme is based on the analysis of small sub-problems with fixed size (up to $\tilde{N} = 10$, independently of N), the computational time required for this analysis nevertheless increases with increasing problem size N, due to the fact that the optimal values of the QAOA depth p tend to increase for larger N. However, the idea of the present paper is rather to analyze the general potential of LR-QAOA, assuming that suitable circuit parameters are already known or can be efficiently found. An important insight provided by our article is that methods based on extrapolation and scaling of circuit parameters with increasing problem size appear to be a promising tool for this purpose, and the results presented here may inspire further development of such methods. Another possible idea, which remains to be explored, is to use the same parameters for different instances belonging to the same class of combinatorial optimization problems.

Probably the most important open topic concerns the question whether quantum optimization admits a polynomial-time solution of some combinatorial optimization problems which, classically, can be solved only in exponential time. To answer this question, a larger range of problem sizes (beyond $N = 28$ as in the present paper) must be addressed in future work, which will either require extensive quantum simulations using high-performance computers and advanced simulation techniques or real, fault-tolerant quantum hardware with suf-

ficient number of qubits (about 50 to 100 or more). Finally, a deeper understanding of the structures of combinatorial optimization problems which determine the hardness of their solution by classical or quantum methods constitutes another challenging goal for future research.

ACKNOWLEDGMENT

This work is funded by the Ministry of Economic Affairs, Labour and Tourism Baden-Württemberg in the frame of the Competence Center Quantum Computing Baden-Württemberg. A special thanks goes to Michael Krebsbach for his valuable support.

REFERENCES

[1] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm (2014), arXiv:1411.4028 [quant-ph].

[2] E. Farhi, J. Goldstone, S. Gutmann, and L. Zhou, The Quantum Approximate Optimization Algorithm and the Sherrington-Kirkpatrick Model at Infinite Size, Quantum **6**, 759 (2022).

[3] C. Grange, M. Poss, and E. Bourreau, An introduction to variational quantum algorithms for combinatorial optimization problems, 4OR **21**, 363–403 (2023).

[4] K. Blekos, D. Brand, A. Ceschini, C.-H. Chou, R.-H. Li, K. Pandya, and A. Summer, A review on quantum approximate optimization algorithm and its variants, Physics Reports **1068**, 1 (2024).

[5] R. Shaydulin and Y. Alexeev, Evaluating quantum approximate optimization algorithm: A case study, in *2019 Tenth International Green and Sustainable Computing Conference (IGSC)* (IEEE, 2019) p. 1–6.

[6] S. H. Sureshbabu, D. Herman, R. Shaydulin, J. Basso, S. Chakrabarti, Y. Sun, and M. Pistoia, Parameter Setting in Quantum Approximate Optimization of Weighted Problems, Quantum **8**, 1231 (2024).

[7] V. Kremenetski, T. Hogg, S. Hadfield, S. J. Cotton, and N. M. Tubman, Quantum alternating operator ansatz (QAOA) phase diagrams and applications for quantum chemistry (2021), arXiv:2108.13056 [quant-ph].

[8] J. Sud, S. Hadfield, E. Rieffel, N. Tubman, and T. Hogg, Parameter-setting heuristic for the quantum alternating operator ansatz, Phys. Rev. Res. **6**, 023171 (2024).

[9] F. G. S. L. Brandao, M. Broughton, E. Farhi, S. Gutmann, and H. Neven, For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances (2018), arXiv:1812.04170 [quant-ph].

[10] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices, Phys. Rev. X **10**, 021067 (2020).

[11] S. H. Sack and M. Serbyn, Quantum annealing initialization of the quantum approximate optimization algorithm, Quantum **5**, 491 (2021).

[12] V. Kremenetski, A. Apte, T. Hogg, S. Hadfield, and N. M. Tubman, Quantum alternating operator ansatz (QAOA) beyond low depth with gradually changing unitaries (2023), arXiv:2305.04455 [quant-ph].

[13] R. Sakai, H. Matsuyama, W.-H. Tam, Y. Yamashiro, and K. Fujii, Linearly simplified QAOA parameters and transferability (2024), arXiv:2405.00655 [quant-ph].

[14] R. Shaydulin, S. Hadfield, T. Hogg, and I. Safro, Classical symmetries and the quantum approximate optimization algorithm, Quantum Information Processing **20**, 359 (2021).

[15] G. B. Mbeng, R. Fazio, and G. Santoro, Quantum annealing: a journey through digitalization, control, and hybrid quantum variational schemes (2019), arXiv:1906.08948 [quant-ph].

[16] G. E. Crooks, Performance of the quantum approximate optimization algorithm on the maximum cut problem (2018), arXiv:1811.08419 [quant-ph].

[17] J. A. Montanez-Barrera and K. Michielsen, Towards a universal QAOA protocol: Evidence of a scaling advantage in solving some combinatorial optimization problems (2024), arXiv:2405.09169 [quant-ph].

[18] S. Boulebnane and A. Montanaro, Solving boolean satisfiability problems with the quantum approximate optimization algorithm, PRX Quantum **5**, 030348 (2024).

[19] R. Shaydulin, C. Li, S. Chakrabarti, M. DeCross, D. Herman, N. Kumar, J. Larson, D. Lykov, P. Minssen, Y. Sun, Y. Alexeev, J. M. Dreiling, J. P. Gaebler, T. M. Gatterman, J. A. Gerber, K. Gilmore, D. Gresh, N. Hewitt, C. V. Horst, S. Hu, J. Johansen, M. Matheny, T. Mengle, M. Mills, S. A. Moses, B. Neyenhuis, P. Siegfried, R. Yalovetzky, and M. Pistoia, Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem, Science Advances **10**, eadm6761 (2024).

[20] F. Glover, G. Kochenberger, and Y. Du, A tutorial on formulating and using QUBO models (2019), arXiv:1811.11538 [cs.DS].

[21] H. Markowitz, Portfolio selection, The Journal of Finance **7**, 77 (1952).

[22] S. Brandhofer, D. Braun, V. Dehn, G. Hellstern, M. Hüls, Y. Ji, I. Polian, A. S. Bhatia, and T. Wellens, Benchmarking the performance of portfolio optimization with QAOA, Quantum Information Processing **22**, 25 (2022).

[23] G. Hellstern, V. Dehn, and M. Zaefferer, Quantum computer based feature selection in machine learning, IET Quantum Communication **5**, 232 (2024).

[24] H. Hofmann, Statlog (German Credit Data), UCI Machine Learning Repository (1994), DOI: https://doi.org/10.24432/C5NC77.

[25] S. Lloyd, Least squares quantization in pcm, IEEE transactions on information theory **28**, 129 (1982).

[26] R. Sibson, SLINK: an optimally efficient algorithm for the single-link cluster method, The Computer Journal **16**, 30 (1973).

[27] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *et al.*, A density-based algorithm for discovering clusters in large spatial databases with noise., in *kdd*, Vol. 96 (1996) pp. 226–231.

[28] J. Saiphet, S. Suwanna, T. Chotibut, and A. Chantasri, Quantum approximate optimization and k-means algorithms for data clustering, Journal of Physics: Conference Series **1719**, 012100 (2021).

[29] J. S. Otterbach, R. Manenti, N. Alidoust, A. Bestwick, M. Block, B. Bloom, S. Caldwell, N. Didier, E. S. Fried, S. Hong, P. Karalekas, C. B. Osborn, A. Papageorge,

E. C. Peterson, G. Prawiroatmodjo, N. Rubin, C. A. Ryan, D. Scarabelli, M. Scheer, E. A. Sete, P. Sivarajah, R. S. Smith, A. Staley, N. Tezak, W. J. Zeng, A. Hudson, B. R. Johnson, M. Reagor, M. P. da Silva, and C. Rigetti, Unsupervised machine learning on a hybrid quantum computer (2017), arXiv:1712.05771 [quant-ph].

[30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research **12**, 2825 (2011).

[31] I. I. Cplex, V12. 1: User's manual for cplex, International Business Machines Corporation **46**, 157 (2009).

[32] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual (2023).

[33] I. Dunning, S. Gupta, and J. Silberholz, What works best when? a systematic evaluation of heuristics for max-cut and QUBO, INFORMS Journal on Computing **30** (2018).

[34] M. X. Goemans and D. P. Williamson, Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming, J. ACM **42**, 1115–1145 (1995).

[35] Qiskit contributors, Qiskit: An open-source framework for quantum computing (2023).

Appendix A: Additional classical results

Fig. 13 illustrates the runtime for the clustering problem using the Blobs dataset. Employing a different dataset results in slightly varying outcomes. This emphasizes the influence of both the dataset and the selected instances on runtime scaling. Notably, the Goemans-Wiliamson algorithm is able to find the optimal solution in this case, and exhibits a better scaling factor α (slope of exponential fit) compared to the Moons dataset.

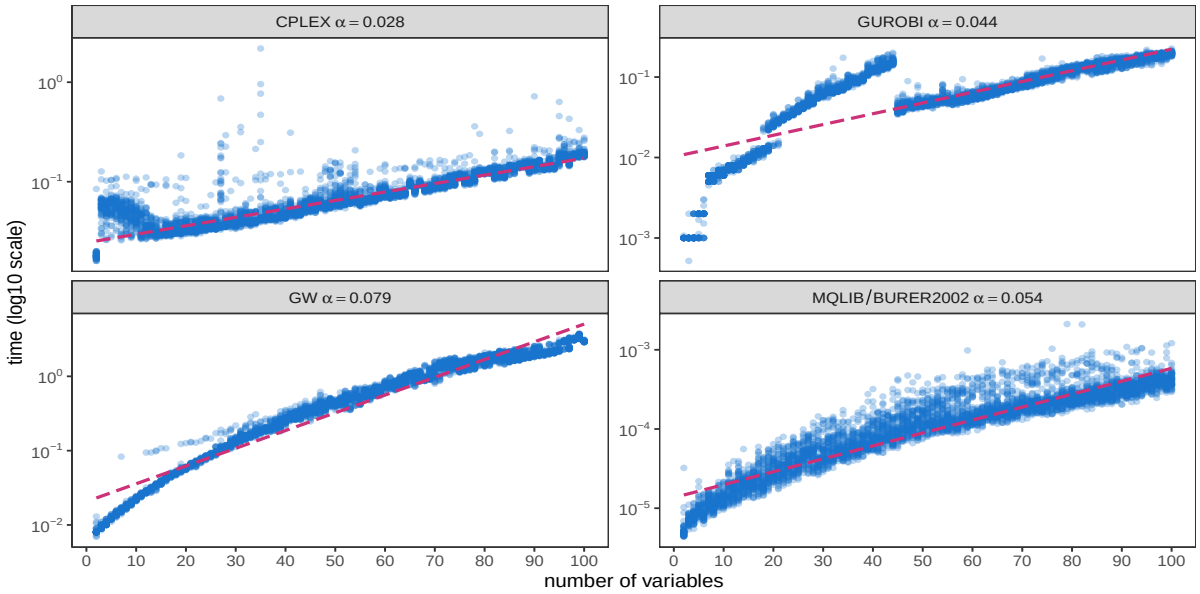

FIG. 13. Clustering (Blobs): Runtime T (seconds) obtained with CPLEX, Gurobi, Goemans-Williamson and MQLib/Burer2002, as a function of the number n of variables (bits). The dots represent the results for the ten problem instances, with blue indicating it was optimally solved, while orange shows an approximate final solution. The slope α of a robust log2-linear model, i.e., $T \propto 2^{\alpha n}$ (red dashed line) is denoted in the header of each figure.

Appendix B: LR-QAOA parameters for feature selection and clustering

Fig. 14 shows the same parameters for the feature selection problem as shown in the main text (Fig. 11) for portfolio optimization. In this case, Δ_γ and Δ_β remain relatively constant as the problem size increases.
The parameters for the clustering problem are illustrated in Fig. 15, again showing a different trend compared to the other problems. Here, we see Δ_γ decreasing and Δ_β increasing.

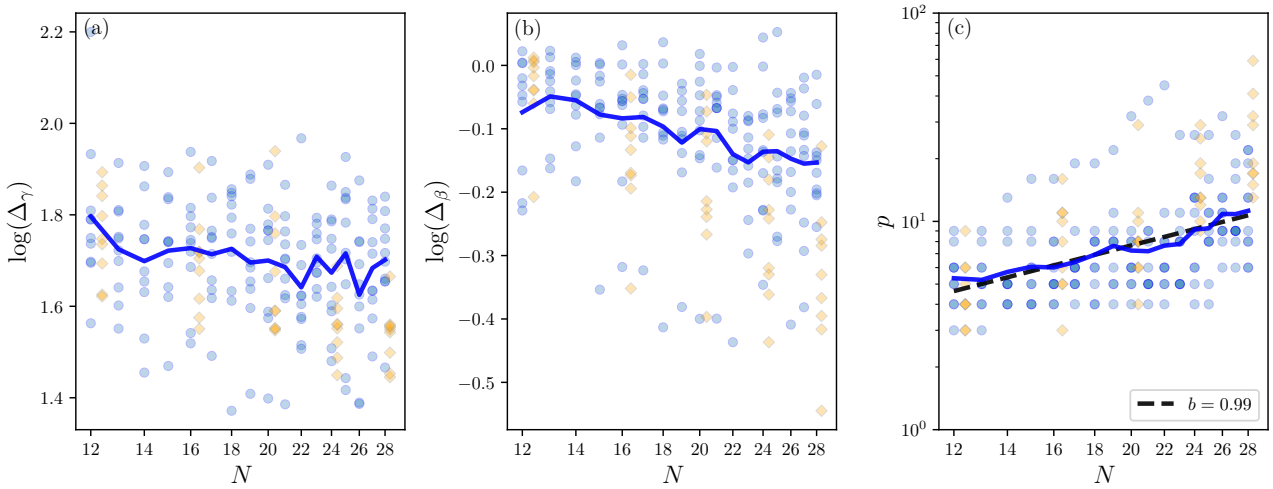

FIG. 14. Feature Selection: same representation as for portfolio optimization of optimized linear-ramp parameters (a): Δ_γ , (b): Δ_β, and (c): QAOA depth p as a function of number of qubits N on a log-log scale. In this case, the logarithmic values for Δ_γ and for Δ_β are relatively constant with N. For the depth p, we observe a nearly linear scaling with respect to N. The yellow diamonds again represent the optima found by a minimizer.

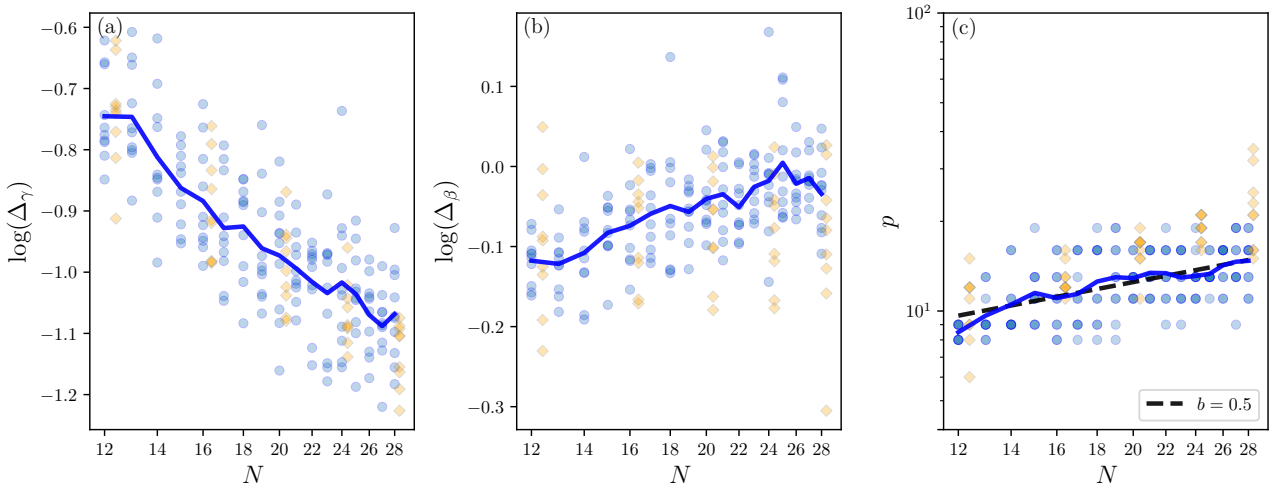

FIG. 15. Clustering (Moons): optimized linear-ramp parameters (a): Δ_γ , (b): Δ_β, and (c): QAOA depth p as a function of number of qubits N on a log-log scale. In this case, the values for Δ_γ are decreasing while the values for Δ_β are slightly increasing with N. For the depth p, we observe a scaling of $\propto \sqrt{N}$. The optima found by a minimizer are represented by the yellow diamonds.